Il problema della classificazione a tre classi ($K = 3$)

I classificatori di base (come la Regressione Logistica standard o le Support Vector Machine originarie) sono **intrinsecamente binari**: sanno rispondere solo a domande del tipo "Sì/No" (es. Classe 1 o Classe 0).
Quando il problema prevede tre classi (es. $C_1 =$ Triangolo, $C_2 =$ Quadrato, $C_3 =$ Cerchio), non possiamo usare un singolo modello binario. L'intuizione principale è **scomporre il problema multiclasse in una serie di problemi binari più semplici**.
Esistono due modi principali per farlo: usare 3 classificatori contro il resto (One-vs-All) o 3 classificatori a coppie (One-vs-One).

L'approccio One-vs-All (OvA)

L'idea del metodo One-vs-All (o One-vs-Rest) è quella di isolare una classe alla volta, raggruppando temporaneamente tutte le altre in un'unica macro-categoria "Altro". Per $K = 3$ classi, addestriamo esattamente **3 classificatori binari distinti**:

1. **Classificatore** $h_1(x)$: Impara a distinguere C_1 da tutto il resto ($C_2 \cup C_3$). Risponde alla domanda "è un Triangolo oppure no?".
2. **Classificatore** $h_2(x)$: Impara a distinguere C_2 da tutto il resto ($C_1 \cup C_3$). Risponde alla domanda "è un Quadrato oppure no?".
3. **Classificatore** $h_3(x)$: Impara a distinguere C_3 da tutto il resto ($C_1 \cup C_2$). Risponde alla domanda "è un Cerchio oppure no?".
   In generale, ogni $h_\theta(x)$ la scriveremo come segue:

$$h_\theta(x) = P(y = triangolo | [x_1, x_2]; \theta) \in [0, 1]$$

   Fase di predizione: Quando arriva un nuovo dato x, lo diamo in pasto a tutti e tre i classificatori. Ognuno restituirà una probabilità (o un punteggio di confidenza) di appartenenza alla propria classe. La regola di decisione standard è il **criterio del massimo (Argmax)**: vince la classe il cui classificatore è più "sicuro".

$$\text{Classe Predetta} = \text{argmax}_{k \in \{1,2,3\}} \, h_k(x)$$

Il problema delle Zone di Ambiguità (Reject Region)

Sebbene intuitivo, l'approccio OvA ha un difetto geometrico e probabilistico strutturale. Poiché i 3 classificatori sono addestrati in modo **completamente indipendente l'uno dall'altro**, le loro probabilità non sono coordinate e non sommano a 1.
Questo genera delle pericolose **zone di ambiguità** nello spazio delle decisioni.
I tre scenari di ambiguità in OvA:

1. **Rifiuto collettivo (Nessuno vince):** Tutti e 3 i classificatori restituiscono una probabilità molto bassa (es. $h_1 = 0.2$, $h_2 = 0.3$, $h_3 = 0.1$). Il modello non sa che decisione prendere. Applicare l'argmax qui significherebbe forzare C_2 (che ha 0.3) con pochissima confidenza.
2. **Rivendicazione multipla (Più vincitori):** Due classificatori restituiscono entrambi una probabilità > 0.5 (es. $h_1 = 0.8$ e $h_2 = 0.7$). Teoricamente impossibile nella realtà, ma possibile matematicamente perché h_1 e h_2 sono modelli separati.
3. **Dominio asimmetrico:** Le classi originali potrebbero essere sbilanciate. Quando si crea il raggruppamento "Altro", la classe negativa diventa molto più numerosa della classe positiva, creando problemi di sbilanciamento (class imbalance) su ogni singolo classificatore binario. Questo è un problema più serio di quello che sembra: anche con dataset bilanciati (es. 100 Triangoli, 100 Quadrati e 100 Cerchi), quando consideriamo un classificatore binario avremo 100 Triangoli e 200 oggetti "Altro". Man mano che andiamo avanti così, per esempio con 10 classi distinte, ci ritroveremo con il 90% di oggetti "Altro". Significa che, se il classificatore rispondesse sempre in maniera negativa (ovvero dicendo "questo oggetto non è un Triangolo") avrebbe ragione il 90% delle volte. Il modello impara un **trucco estremamente comodo a livello statistico**: conviene sempre rispondere no. Si tratta di un serio problema di **Bias** (nel senso di "propensione sistematica") e di sbilanciamento.
   Qualora nessuna probabilità riuscisse a superare una soglia minima di certezza (es. $P < 0.7$), sono possibili due strategie:

- Forzare una scelta (prendendo il massimo) in task non critici.
- **Reject Option:** scartare l'input ("non so") in contesti delicati (es. diagnostica).
  Spesso per l'approccio OvA si parla di **zone d'ombra**, che si suddividono in due zone distinte:
- **Zone di rifiuto:** poiché i modelli non comunicano tra loro, ognuno deciderà dove tirare la propria retta per separate la propria classe da tutto il resto, e questa retta sarà completamente indipendente dalle rette degli altri classificatori. Ciò crea problemi quando un dato si trova in un'area dello spazio lontana da tutti gli esempi chiari di training. A questo punto, tutti i modelli daranno una probabilità molto bassa per questo dato, ma con l'argmax andremmo comunque a scegliere quello con la probabilità più

alta (che ricordiamo essere una probabilità bassa, come $0,20$). Questa è una decisione estremamente fragile, che porterà a errori quasi certi.

- **Zone di conflitto:** un problema analogo al precedente si verifica quando un dato si trova in una zona di confine sfumata tra due o più classi, e si potrebbe verificare che tutti i classificatori restituiscano una probabilità molto alta per quel singolo dato. Questo ci porterà comunque a scegliere la probabilità maggiore (es. $0,85$), ma ignoreremo che gli altri due classificatori avevano comunque generato delle probabilità molto alte (es. $0,80$).

Considerazione sulle probabilità restituite dai tre modelli

Supponiamo che il modello $h_1(x)$ restituisca probabilità pari a $0,2$ per la classe triangolo. Significa che la probabilità per quadrato + cerchio è pari a $0,8$. Consideriamo ora il modello $h_2(x)$, che restituisce la probabilità per la classe quadrato, questa sarà pari a $0,8 - h_3(x)$ e allo stesso modo il modello $h_3(x)$ restituirà una probabilità pari a $0,8 - h_2(x)$. Sembrano calcoli ragionevoli da fare, tuttavia non è così, e sono sbagliati. I tre modelli utilizzati **sono fisicamente separati, e ciechi l'uno rispetto all'altro**: si tratta di classificatori costruiti in modo diverso.
Questa logica, però, ci tornerà utile per la **Regressione Softmax**, che vedremo più avanti.

L'alternativa One-vs-One (OvO)

Per evitare il problema dello sbilanciamento artificiale del raggruppamento "Altro", si può usare l'approccio a coppie. Per $K = 3$ classi, si addestrano classificatori binari per **ogni possibile coppia**.
Il numero di classificatori necessari è $\frac{K(K-1)}{2}$. Per $K = 3$, la formula dà di nuovo **3 classificatori**:

1. **Classificatore 1:** C_1 vs C_2 (Ignora totalmente i dati di C_3 durante il training).
2. **Classificatore 2:** C_1 vs C_3 (Ignora C_2).
3. **Classificatore 3:** C_2 vs C_3 (Ignora C_1).
   Fase di predizione (Voting a maggioranza): il nuovo dato x viene valutato dai 3 classificatori. Ognuno esprime un "voto" netto per una delle due classi che conosce. La classe che ottiene più voti vince (es. se C_1 vince contro C_2 e vince anche contro C_3, totalizza 2 voti e viene scelta).

Pro e Contro di OvO

- **Pro:** I confini di decisione sono più netti e non si crea il problema dello sbilanciamento delle classi (i dati di training per ogni modello sono solo quelli delle due classi interessate).
- **Contro:** Se il numero di classi cresce (es. $K = 10$), il numero di classificatori esplode ($\frac{10 \times 9}{2} = 45$). Inoltre, può persistere un'ambiguità in caso di "stallo" nei voti (es. C_1 batte C_2, C_2 batte C_3, C_3 batte C_1).

Valutazione di un classificatore: la matrice di confusione

I problemi dell'accuracy

Immaginiamo un test set con 100 campioni di classe negativa e soli 2 campioni di classe positiva (classi molto sbilanciate). Un modello che classificasse tutto come negativo otterrebbe comunque:

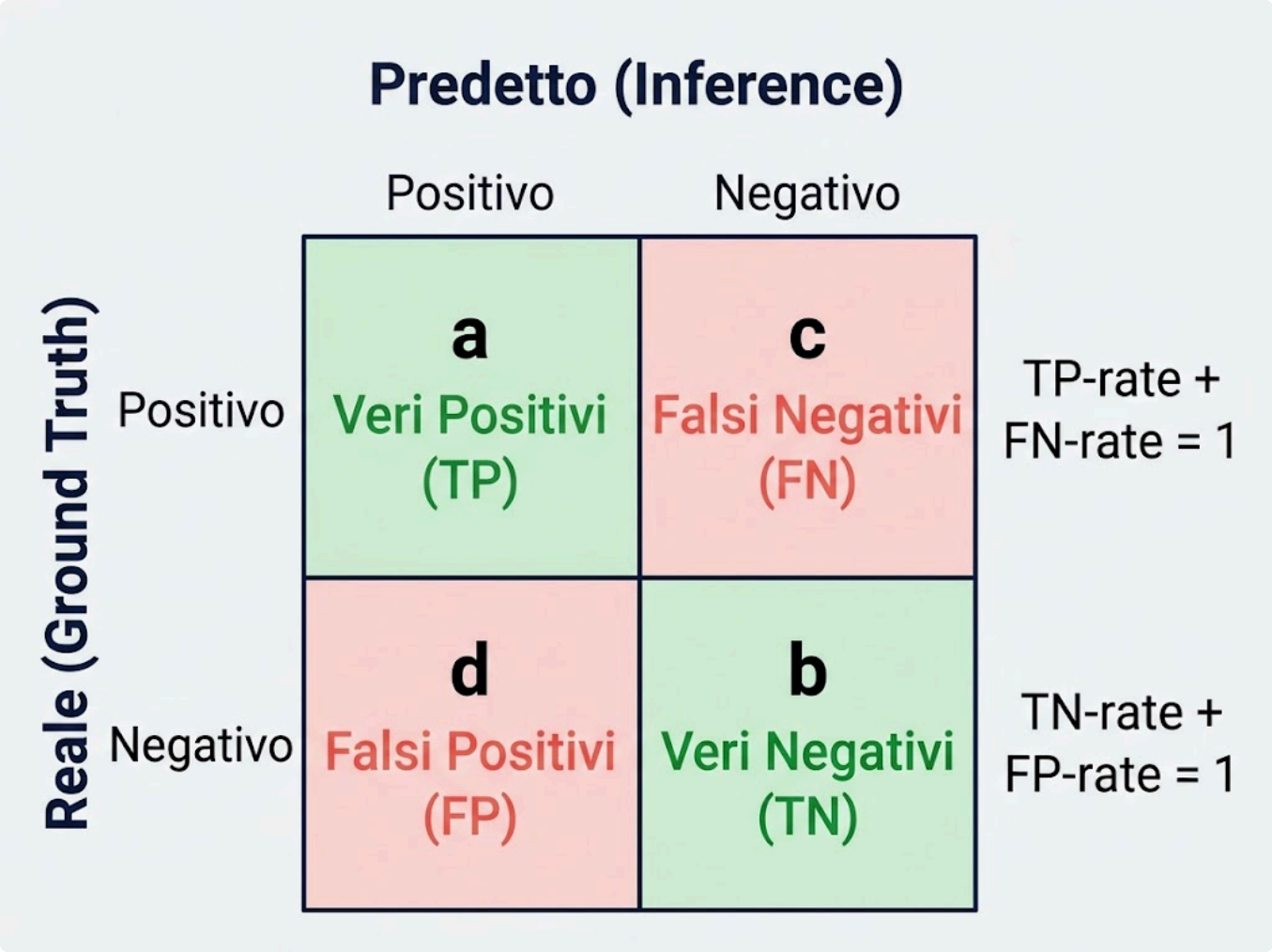
$$\text{Accuracy} = \frac{100}{102} \approx 98\%$$

pur non avendo mai riconosciuto correttamente un solo caso positivo. Per questo motivo l'accuracy va sempre affiancata da altre metriche, costruite a partire dalla **matrice di confusione**, e i test set vanno costruiti (quando possibile) in modo bilanciato tra le classi.

La matrice di confusione (caso binario)

Per un problema binario (positivo / negativo) la matrice di confusione mette in relazione l'etichetta reale (righe) con l'etichetta predetta dal modello (colonne). In altri contesti le etichette potrebbero essere invertite, ma in questo corso si seguirà la convenzione

spiegata poco sopra.



Metriche derivate: Accuracy, Precision e Recall

- **Accuracy**, percentuale di campioni classificati correttamente:

$$\text{Accuracy} = \frac{a+b}{a+b+c+d}$$

- **Recall / Sensitivity**, di tutti i casi realmente positivi, indica quale frazione viene individuata:

$$\text{Recall} = \frac{a}{a+c} = \frac{TP}{TP+FN}$$

- **True Negative Rate (Specificity)**:

$$\text{Specificity} = \frac{b}{b+d} = \frac{TN}{TN+FP}$$

- **Precision**, di tutti i casi predetti come positivi, indica quanti lo sono davvero:

$$\text{Precision} = \frac{a}{a+d} = \frac{TP}{TP+FP}$$

Nota: la Precision e la Recall rispondono a domande diverse: la **Recall** penalizza i falsi negativi (vogliamo "beccarli tutti"), la **Precision** penalizza i falsi positivi (vogliamo essere sicuri quando diciamo "positivo"). In un test medico per una malattia grave si preferisce quasi sempre un'alta Recall.

- **F1-score** (media armonica tra Precision e Recall, utile quando le metriche sono sbilanciate tra loro):

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Perché la media armonica? Perché punisce fortemente gli squilibri. Esempio con tre modelli:

Modello	Precision	Recall	Media aritmetica	F1-score
Modello 1	0.50	0.40	0.45	0.44
Modello 2	0.70	0.10	0.40	0.175
Modello 3	0.02	1.00	0.51	0.039

Il Modello 3 ha una media aritmetica alta (0.51) pur avendo una Precision quasi nulla (il modello dice sempre "positivo"). L'F1-score lo penalizza correttamente (0.039). **Casi limite**: se Precision = 0 oppure Recall = 0 $\rightarrow$ F1 = 0. Se Precision = Recall = 1 $\rightarrow$ F1 = 1.

- **F-beta score** (generalizzazione dell'F1-score che utilizza un parametro β per dare maggiore o minore importanza alla Recall rispetto alla Precision):

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{(\beta^2 \cdot \text{Precision}) + \text{Recall}}$$

Come impostare il parametro β:

1. Se $\beta = 1$: Precision e Recall hanno lo stesso peso (la formula si riduce all'esatto F1-score).
2. Se $\beta > 1$ (es. $\beta = 2$): si dà più peso alla **Recall** (utile quando i Falsi Negativi sono molto pericolosi, come in ambito medico).
3. Se $\beta < 1$ (es. $\beta = 0.5$): si dà più peso alla **Precision** (utile quando i Falsi Positivi sono costosi, come nello spam filtering).

La soglia di decisione e il trade-off Precision/Recall

Un classificatore come la regressione logistica restituisce una probabilità $h_\theta(x) \in [0, 1]$. Per ottenere un'etichetta serve una soglia (di solito 0.5). Cambiare la soglia comporterà un cambiamento della precision e della recall, ma in versi opposti:

- **Soglia più alta:** il modello è più "esigente", sale la Precision, scende la Recall.
- **Soglia più bassa:** il modello dice "positivo" più facilmente, sale la Recall, scende la Precision.

Curva Precision-Recall (PR Curve)

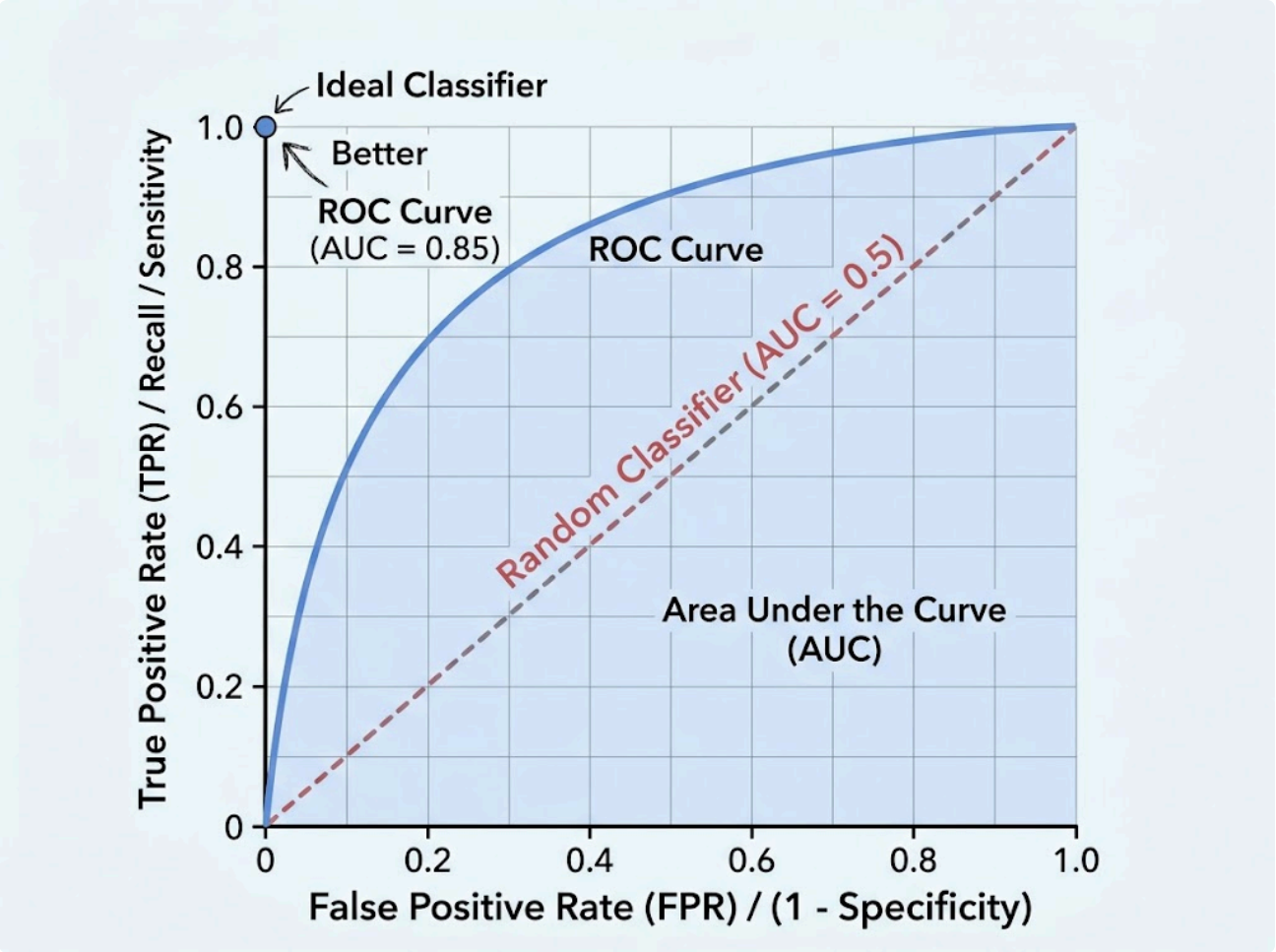
Mette in relazione la **Recall** (sull'asse X) e la **Precision** (sull'asse Y). Mostra graficamente il trade-off tra le due metriche: quanto si è disposti a sacrificare in precisione pur di trovare tutti i casi positivi (Recall al 100%)?

- **Quando si usa:** è l'ideale quando il dataset è fortemente sbilanciato (es. ricerca di una malattia rara), perché ignora i TN (che in un set sbilanciato "gonfiano" le statistiche in modo fuorviante).

Curva ROC (Receiver Operating Characteristic)

Mette in relazione il **False Positive Rate** (FPR, asse X) e il **True Positive Rate** (TPR o Recall, asse Y). Mostra il compromesso tra il tasso di veri positivi e il tasso di falsi allarmi.

- **Quando si usa:** è lo standard per dataset bilanciati.
- **L'Area Under the Curve (AUC):** l'area sottesa alla curva ROC è una metrica di sintesi eccezionale. Un'AUC pari a 0.5 equivale a tirare una moneta (classificatore inutile), mentre un'AUC di 1.0 indica un modello perfetto che separa le classi senza alcun errore.



Estensione al caso multiclasse (K classi)

Con K classi, la matrice diventa $K \times K$: l'elemento $_{i,j}$ è il numero di campioni di classe reale i classificati come classe „ Per ogni classe k:

- $TP_k = _{k,k}$
- $FP_k = _{i_k\ i,k}$ (somma colonna k, tolta diagonale)
- $FN_k = _{i_k\ k,i}$ (somma riga k, tolta diagonale)

- $TN_k = (_,_) - TP_k - FP_k - FN_k$
  Si calcolano Precision, Recall e F1 per ogni classe e se ne fa la media (**macro-average**). L'accuracy complessiva è la somma della diagonale diviso il totale.

Dataset e relazioni con Overfitting, Underfitting e Regolarizzazione

Quando ha senso aumentare i dati?

- **Underfitting (High Bias):** il modello è troppo semplice, commette errori sistematici su train e test set. Aumentare i dati non aiuta molto, serve un modello più espressivo.
- **Overfitting (High Variance):** il modello è troppo complesso, si adatta "a memoria" al training set (incluso il rumore) e generalizza male. Aumentare i dati in questo caso può aiutare.

Overfitting nella classificazione: regolarizzazione Ridge

Il rischio di overfitting si riduce aggiungendo un termine di regolarizzazione 2 alla funzione di costo (es. Binary Cross-Entropy):

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left|y^{(i)}\log(h_\theta(x^{(i)})) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))\right| + \frac{\lambda}{2m}\sum_{j=1}^{n}\theta_j^2$$

Il termine $\lambda\ \theta_j^2$ "penalizza" i pesi grandi, spingendo il modello verso ipotesi più semplici (vicine a $\theta \approx 0$). Ovviamente, il bias θ_0 non viene penalizzato in quanto non vogliamo limitare in alcun modo l'intercetta della retta.

Train / Validation / Test e scelta degli iperparametri

Perché training e test non bastano

Se scegliessimo gli iperparametri guardando l'errore sul test set, staremmo implicitamente "allenando" anche gli iperparametri su quei dati. Si usa un terzo set:

1. **Train:** impara i parametri θ minimizzando $J(\theta)$.

2. **Validation:** sceglie gli iperparametri (λ, grado polinomio, ecc.) minimizzando J_{val}.
3. **Test:** stima l'errore di generalizzazione finale (usato una sola volta).
   Le 3 funzioni di costo per la valutazione (senza termine λ):

$$J_{train}(\theta) = \frac{1}{2m_{train}} \sum (h_\theta(x^{(i)}_{train}) - y^{(i)}_{train})^2$$

$$J_{val}(\theta) = \frac{1}{2m_{val}} \sum (h_\theta(x^{(i)}_{val}) - y^{(i)}_{val})^2$$

$$J_{test}(\theta) = \frac{1}{2m_{test}} \sum (h_\theta(x^{(i)}_{test}) - y^{(i)}_{test})^2$$

K-Fold Cross Validation

Quando i dati sono pochi, per non "sprecare" un validation set fisso:

1. Si separa da subito un **test set** (es. 20%) intoccabile.
2. L'80% rimanente viene diviso in K **fold**.
3. Per ogni combinazione di iperparametri H:
   - Per ogni $k = 1...K$: allena sui restanti $K-1$ fold, valuta sul fold $k \rightarrow J^H_{val}(\theta^{(k)})$.
   - Fai la media sui K fold totali del validation (contando ogni iterazione): $\bar{J}^H_{val}(\theta)$.
4. Scegli l'iperparametro H che minimizza l'errore medio (si confrontano le medie di ogni combinazione sui K fold totali).
5. Allena il modello finale e valutalo sul test set.

Scelta del parametro λ (Grid Search)

Si prova una griglia di valori (es. $0, 0.01, 0.02, 0.04, \ldots$):

- λ **grande:** regolarizzazione forte $\rightarrow$ i pesi tendono a 0 $\rightarrow$ Underfitting (High bias)
- λ **piccolo:** poca regolarizzazione $\rightarrow$ Overfitting (High variance)
- **Ottimo:** il punto medio in cui J_{val} è minimo.

Bias vs Varianza (Complessità del modello)

In generale, sappiamo che **Errore = Bias + Varianza + Rumore**. Tralasciando il rumore (che non possiamo eliminare), le prestazioni del modello dipendono da bias e varianza.

- **Underfitting (Grado basso):** J_{train} e J_{val} alti e vicini. Si ha un Bias alto (es. usare una retta di grado 1 per descrivere una curva di punti molto complessa), in quanto si commette un errore sistematico su entrambi i set.
- **Ottimo:** J_{train} e J_{val} bassi e vicini.
- **Overfitting (Grado alto):** J_{train} basso, ma J_{val} resta alto (gap ampio). Si ha una varianza alta, in quanto il modello è troppo flessibile (es. un polinomio di grado troppo alto). Invece di imparare a generalizzare, il modello impara perfettamente il training set, incluso il rumore, senza generalizzare.

Learning Curve (Errore vs Dimensione Training Set)

Facendo variare m (numero di campioni):

- **High bias:** le curve convergono presto a un valore comune ma alto (più dati non aiutano).
- **High variance:** gap ampio e persistente tra J_{train} e J_{val} (più dati possono aiutare).

Tips & Tricks

Problema	Possibili interventi
High variance (overfitting)	Aumentare dati di training; ridurre set di feature; aumentare λ.
High bias (underfitting)	Aumentare set di feature; usare feature polinomiali; diminuire λ.

Ottimizzare la discesa del gradiente

Scegliere il learning rate (α)

- Se $J(\theta)$ oscilla o cresce $\rightarrow$ α troppo grande.

- Se $J(\theta)$ decresce troppo lentamente $\rightarrow \alpha$ troppo piccolo.
- Griglia tipica da provare: $0.001, 0.003, 0.01, 0.03, 0.1, 0.3$.

Momentum

Senza il Momentum, la Discesa del Gradiente rischia di rimbalzare di continuo a destra e a sinistra delle pareti di un minimo, rallentando drasticamente il tempo di convergenza. Questo fenomeno si verifica quasi sempre quando la funzione di costo presenta una topologia a "valle stretta", ovvero un avvallamento dove le pareti laterali sono molto ripide, mentre il fondovalle scende dolcemente verso il minimo.
Il problema dello Zig-Zag (Senza Momentum): in una valle stretta, i vettori del gradiente puntano perpendicolarmente verso le pareti anziché lungo la discesa.

- L'algoritmo esegue un aggiornamento, ma la pendenza laterale è così forte che il passo "scavalca" il centro della valle e finisce sulla parete opposta.
- All'iterazione successiva, il gradiente punta nella direzione opposta e l'algoritmo rimbalza indietro.
- Risultato: il modello spreca la quasi totalità del tempo a oscillare trasversalmente, avanzando verso il minimo a passi microscopici.
  Il vicolo cieco del Learning Rate Per fermare le oscillazioni orizzontali senza usare il Momentum, l'unica soluzione sarebbe abbassare drasticamente il learning rate (α). Tuttavia, un α microscopico costringerà l'algoritmo a compiere passi minuscoli anche nella direzione corretta, allungando a dismisura i tempi di addestramento. Il Momentum interviene annullando matematicamente i vettori laterali discordanti e potenziando la spinta costante verso il minimo.
  Accelera la convergenza smorzando le oscillazioni tramite una media mobile dei gradienti:

$$z_{new} = \beta \cdot z_{old} + \frac{\partial J(\theta_{old})}{\partial \theta_{old}}$$

$$\theta_{new} = \theta_{old} - \alpha \cdot z_{new}$$

  β (momentum parameter) di solito è impostato ≈ 0.90.
  Per avere un'idea intuitiva del Momentum, bisogna immaginare z come la **velocità**. β invece rappresenta quanta memoria conservare.

Learning Rate Schedules

Il **learning rate fisso spesso non funziona bene**. Per questo, conviene farlo variare. A questo proposito, esistono due famiglie di approcci: una per cui si seguono delle regole prefissate e rigide, ovvero il Learning Rate Schedules, e l'altra per cui il learning rate si adatta tramite algoritmi intelligenti, ovvero i Metodi Adattivi. Vediamo di seguito le strategie del Learning Rate Schedules:
A ogni iterazione, tutti i parametri subiscono un aggiornamento con lo stesso identico α.

- **Time-based decay (Decadimento basato sul tempo):**

$$\alpha^t = \alpha^0 \cdot \frac{1}{1 + k \cdot t}$$

  Il learning rate scende in modo continuo e fluido a ogni singola iterazione t. Il parametro k decide quanto velocemente "frenare".
- **Step decay (Decadimento a gradini):** Non è una curva fluida, ma una scala. Il learning rate rimane costante per un tot di epoche (es. 10 epoche), poi viene tagliato di netto (es. dimezzato). È molto usato perché permette al modello di stabilizzarsi prima di rallentare ulteriormente.
- **Exponential decay (Decadimento esponenziale):**

$$\alpha^t = \alpha^0 \cdot e^{-k \cdot t}$$

  Scende in modo drastico all'inizio e poi si stabilizza su valori minuscoli verso la fine, seguendo la curva dell'esponenziale negativo.

Metodi Adattivi (Apprendimento su misura)

I metodi adattivi rivoluzionano il concetto: **abbandonano il learning rate globale**. Invece di avere un solo α per tutto il modello, l'algoritmo calcola dinamicamente un learning rate diverso per ogni singolo parametro.

- **Adagrad (Adaptive Gradient):** Tiene traccia di quanto è stato aggiornato ogni singolo parametro in passato.
  - Ai parametri legati a feature frequenti (che vengono aggiornati di continuo) assegna un learning rate *piccolissimo*, per fare aggiustamenti di precisione.

    - Ai parametri legati a feature rare (dati sparsi, es. una parola rara in un testo) assegna un learning rate *grandissimo*, perché avendo poche occasioni per vederla, quando capita deve impararla in fretta.
- **Adam (Adaptive Moment Estimation):** attualmente uno dei metodi adattivi più usati del Deep Learning. Unisce due concetti: l'intelligenza di **Adagrad** (un α diverso per ogni peso) con l'inerzia del **Momentum** (smorzamento degli zig-zag tramite la media dei gradienti passati). Se non si sa quale ottimizzatore usare in una Rete Neurale, conviene in generale usare Adam.
- **LARS (Layer-wise Adaptive Rate Scaling):** Mentre Adam lavora sul singolo parametro, LARS lavora a blocchi spaziali. È progettato per Reti Neurali gigantesche addestrate su enormi moli di dati (Large Batch Training). Analizza la "grandezza" (norma) dei pesi di un intero **Layer** rispetto ai suoi gradienti, e assegna un learning rate specifico per quello strato. Impedisce che l'aggiornamento "esploda", in quanto se il layer in questione possiede un gradiente molto grande rispetto ai pesi, allora LARS abbasserà molto il learning rate per quello specifico layer.